Esercizio: `instanceof`, `switch` esteso, vs. overloading di metodi

Corso di Laurea in Ingegneria Informatica

Anno accademico 2025/2026

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

RECAP: CHECK DINAMICO DI TIPO

- Tutti i linguaggi OO offrono un qualche operatore o metodo per verificare a run time il tipo di un'istanza
  - Java `instanceof`
  - C# `is`
  - Scala `isInstanceOf[`*`Tipo`*`]`
  - Kotlin `as`
- Spesso, alla verifica si può unire la conversione di tipo (cast):
  - Java `instanceof` *esteso*
  - C# `is` *esteso*
  - Scala `asInstanceOf[`*`Tipo`*`]`
  - Kotlin *smart cast – il tipo è già convertito dentro l'if che lo verifica*

RECAP: `switch` ESTESO CON PATTERN MATCHING

- Switch esteso come istruzione:

Java

```
public boolean equals(Object obj){
 switch(obj){
   case Counter c: return this.val==c.val;
   default: return false;
 }
}
```

- Switch esteso come espressione:

Java

```
public boolean equals(Object obj){
 return switch(obj){
   case Counter c -> this.val==c.val;
   default -> false;
 }
}
```

IL TEMA

- IL TEMA: può succedere di aver bisogno di *distinguere le istanze di una classe da quelle delle sue sottoclassi,* per poterle trattare diversamente
- Se la funzione è un metodo, ci pensa il polimorfismo
- MA se la funzione *non è* un metodo, anzi magari è una funzione statica, come risolviamo?
- **Due strade:**
  - **Check dinamico di tipo (`instanceof` o `switch` con pattern matching)**
  - **Overloading**

UN CASO DI STUDIO

- Si supponga di dover scrivere una funzione *statica* **`decrementa`** che riceva un **`Counter`** per decrementarlo
  - per i **`Counter2`** può farlo facilmente, invocando il metodo **`dec`**
  - per i **`Counter`**, invece, dovrà ricorrere a una «perifrasi», scrivendo un algoritmo ad hoc.
- Ecco che occorre quindi poter distinguere le istanze di una classe più generale (**`Counter`**) da quelle di una sua sottoclasse più specifica (**`Counter2`**)
- **Lo possiamo fare in tre modi:**
  - con **`instanceof`**
  - con **`switch`** esteso con pattern matching
  - ricorrendo a due funzioni overloaded

UN CASO DI STUDIO: SPECIFICHE

- **Funzione `decrementa`: specifiche**
  - riceve un **`Counter`** qualsiasi
  - verifica se esso sia in realtà un **`Counter2`**
  - se sì, deve chiamare **`dec`** → necessario un qualche downcast
  - se no, sfrutta **`reset`**, **`getValue`** e **`inc`** per ottenere l'effetto del decremento per altra via, tramite azzeramento e ciclo di incrementi
    - ATTENZIONE: non si può invece sostituire il contatore, come facemmo in un esercizio, perché l'argomento è passato per valore e quindi il riferimento non può cambiare!

MODO #1: USARE `instanceof`

- Esempio: la funzione **`decrementa`**
  - riceve un **`Counter`** qualsiasi
  - verifica se esso sia in realtà un **`Counter2`**
  - se sì, deve chiamare **`dec`** → <u>necessario un cast</u> (sul sicuro, però)
  - se no, sfrutta **`reset`**, **`getValue`** e **`inc`** per ottenere l'effetto del decremento per altra via, tramite azzeramento e ciclo di incrementi

```
public static void decrementa(Counter c){
  if (c instanceof Counter2) {
       var c2 = (Counter2) c; c2.dec();
  } else {
       int valore = c.getValue();
       c.reset();
       while (--valore >0) c.inc();
  }
}
```

MODO #1: USARE `instanceof`

- Esempio: la funzione **`decrementa`**
  - riceve un **`Counter`** qualsiasi
  - verifica se esso sia in realtà un **`Counter2`**
  - se sì, deve chiamare **`dec`** → <u>necessario un cast</u> (sul sicuro, però)
  - se no, sfrutta **`reset`**, **`getValue`** e **`inc`** per ottenere l'effetto del decremento per altra via, tramite azzeramento e ciclo di incrementi

```
public static void decrementa(Counter c){
  if (c instanceof Counter2 c2) { // operatore evoluto
      c2.dec();
  } else {
      int valore = c.getValue();
      c.reset();
      while (--valore >0) c.inc();
  }
}
```

MODO #1: USARE `instanceof`

```java
public static void decrementa(Counter c){
    if (c instanceof Counter2 c2) {
        c2.dec();
    }
    else {
        int valore = c.getValue();
        c.reset();
        while (--valore >0) {
            c.inc();
        }
    }
}

public static void main(String[] args){
    Counter c1 = new Counter(10);
    System.out.println(c1);
    decrementa(c1);
    System.out.println("c1 decrementato: " + c1);
    //----------------------------
    Counter2 c2 = new Counter2(20);
    System.out.println(c2);
    //----------------------------
    decrementa(c2);
    System.out.println("c2 decrementato: " + c2);
}
```

```
Counter di valore 10
c1 decrementato: Counter di valore 9
Counter di valore 20
c2 decrementato: Counter di valore 19
```

MODO #2: `switch` ESTESO

- Un modo alternativo consiste nell'usare il costrutto **`switch`** evoluto con *pattern matching*
  - notare l'uso della variabile anonima «_» nel secondo caso, in cui non interessa rinominare la variabile sottoposta a cast

```
public static void decrementa(Counter c){
  switch(c) {
      case Counter2 c2 -> c2.dec();
      case Counter   _ -> {
            int valore = c.getValue();
            c.reset();
            while (--valore >0) c.inc();
      }
  }
}
```

MODO #2: `switch` ESTESO

```java
public static void decrementa(Counter c){
    switch(c) {
        case Counter2 c2 -> c2.dec();
        case Counter   _ -> {
            int valore = c.getValue();
            c.reset();
            while (--valore >0) c.inc();
        }
    }
}

public static void main(String[] args){
    Counter c1 = new Counter(10);
    System.out.println(c1);
    decrementa(c1);
    System.out.println("c1 decrementato: " + c1);
    //----------------------------
    Counter2 c2 = new Counter2(20);
    System.out.println(c2);
    //----------------------------
    decrementa(c2);
    System.out.println("c2 decrementato: " + c2);
```

```
Counter di valore 10
c1 decrementato: Counter di valore 9
Counter di valore 20
c2 decrementato: Counter di valore 19
```

MODO #3: OVERLOADING

- Un terza possibilità è prevedere fin dall'inizio *due varianti* della funzione **decrementa**, con argomento di tipo diverso
  - evita di dover distinguere i due casi *dentro* alla funzione *perché li distingue già fuori: smart!* ☺

```
public static void decrementa(Counter2 c2){
     c2.dec();
}

public static void decrementa(Counter c){
     int valore = c.getValue();
     c.reset();
     while (--valore >0) {
            c.inc();
     }
}
```

MODO #3: OVERLOADING

```java
public static void decrementa(Counter2 c2){
    c2.dec();
}

public static void decrementa(Counter c){
    int valore = c.getValue();
    c.reset();
    while (--valore >0) {
        c.inc();
    }
}

public static void main(String[] args){
    Counter c1 = new Counter(10);
    System.out.println(c1);
    decrementa(c1);
    System.out.println("c1 decrementato: " + c1);
    //----------------------------
    Counter2 c2 = new Counter2(20);
    System.out.println(c2);
    //----------------------------
    decrementa(c2);
    System.out.println("c2 decrementato: " + c2);
```

```
Counter di valore 10
c1 decrementato: Counter di valore 9
Counter di valore 20
c2 decrementato: Counter di valore 19
```

MODO #3: perché non per `equals`...?

- Visto che di tratta di una possibilità interessante, ci si potrebbe chiedere *perché non usarla anche per* **`equals`**, che abbiamo implementato solo nei modi #1 e #2
- Il motivo è che *non funzionerebbe,* perché *la risoluzione dell'overloading avviene staticamente, a compile time:* a **`equals`** serve invece un cast *dinamico*, a *runtime*
  - in altri termini, in presenza di argomento formale **`Object`** verrebbe sempre selezionata la **`equals`** con argomento **`Object`**, *anche ove questi fosse in realtà un* **`Counter`**
- Verifichiamolo con un esperimento.

MODO #3: perché non per `equals`...?

- Ecco **`equals`** implementata con due varianti overloaded

```
public boolean equals(Object c){
        return false;
}
public boolean equals(Counter that){
        return this.val == that.val;
}
```

- Nel solito main di prova, il caso critico è quello ibrido-mix:

```
System.out.println("c1 equals c2: " + c1.equals(c2));  // true
System.out.println("c2 equals c1: " + c2.equals(c1));  // true

Object obj = c1;

System.out.println("obj(c1) equals c2: " + obj.equals(c2)); // false anziché true
System.out.println("c2 equals obj(c1): " + c2.equals(obj)); // false anziché true
```

```
c1 equals c2: true
c2 equals c1: true
obj(c1) equals c2: false
c2 equals obj(c1): false
```

MODO #3: perché `equals` non funziona

- In **`c2.equals(obj)`** viene chiamata la **`equals`** di **`Counter`** con argomento **`Object`**, che ovviamente ritorna **`false`**
- In **`obj.equals(c2)`** il problema è più subdolo
  - poiché il target **`obj`** è formalmente un **`Object`** e il polimorfismo sceglie fra le funzioni *già definite nella classe del target*, l'unica opzione è la **`equals`** di **`Object`**, *che ha argomento **`Object`***
  - solo dopo aver scelto la signature della funzione da chiamare se ne cerca la versione specifica nella classe **`Counter`**, perché il target è appunto un **`Counter`**: ma in **`Counter`**, la **`equals`** con argomento **`Object`** restituisce **`false`**!
  - la **`equals`** con argomento **`Counter`** non viene neanche vista! ☹

```
System.out.println("obj(c1) equals c2: " + obj.equals(c2)); // false anziché true
System.out.println("c2 equals obj(c1): " + c2.equals(obj)); // false anziché true
```